



Triangulacja wielokąta prostego – porównanie metod

Projekt nr 7

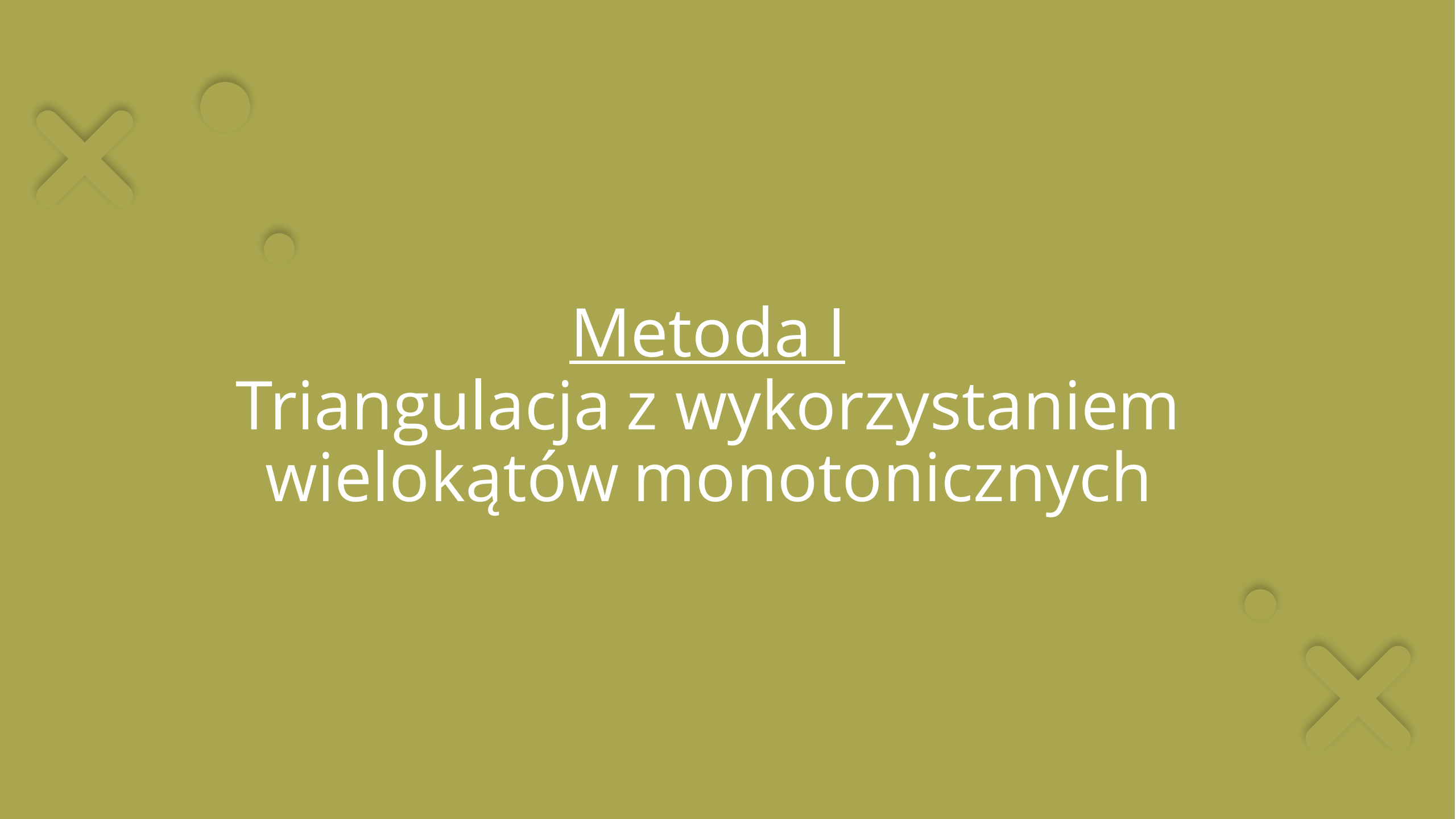
Robert Mesek

Wojciech Kaźmierczak

Grupa 4

Czym jest triangulacja?

- + Triangulacja wielokąta w 2D to podział wielokąta na trójkąty, których jedynymi wierzchołkami są wierzchołki wielokąta.
- + Istnieje wiele triangulacji tego samego wielokąta, jednak każda z nich posiadać będzie tą samą liczbę trójkątów.
- + Dla każdego wielokąta można znaleźć jego triangulację
- + Istnieje wiele różnych metod/algorytmów triangulacji, jednak my dzisiaj zajmiemy się dwoma metodami: Triangulacja z wykorzystaniem wielokątów monotonicznych, Triangulacja Delaunay'a



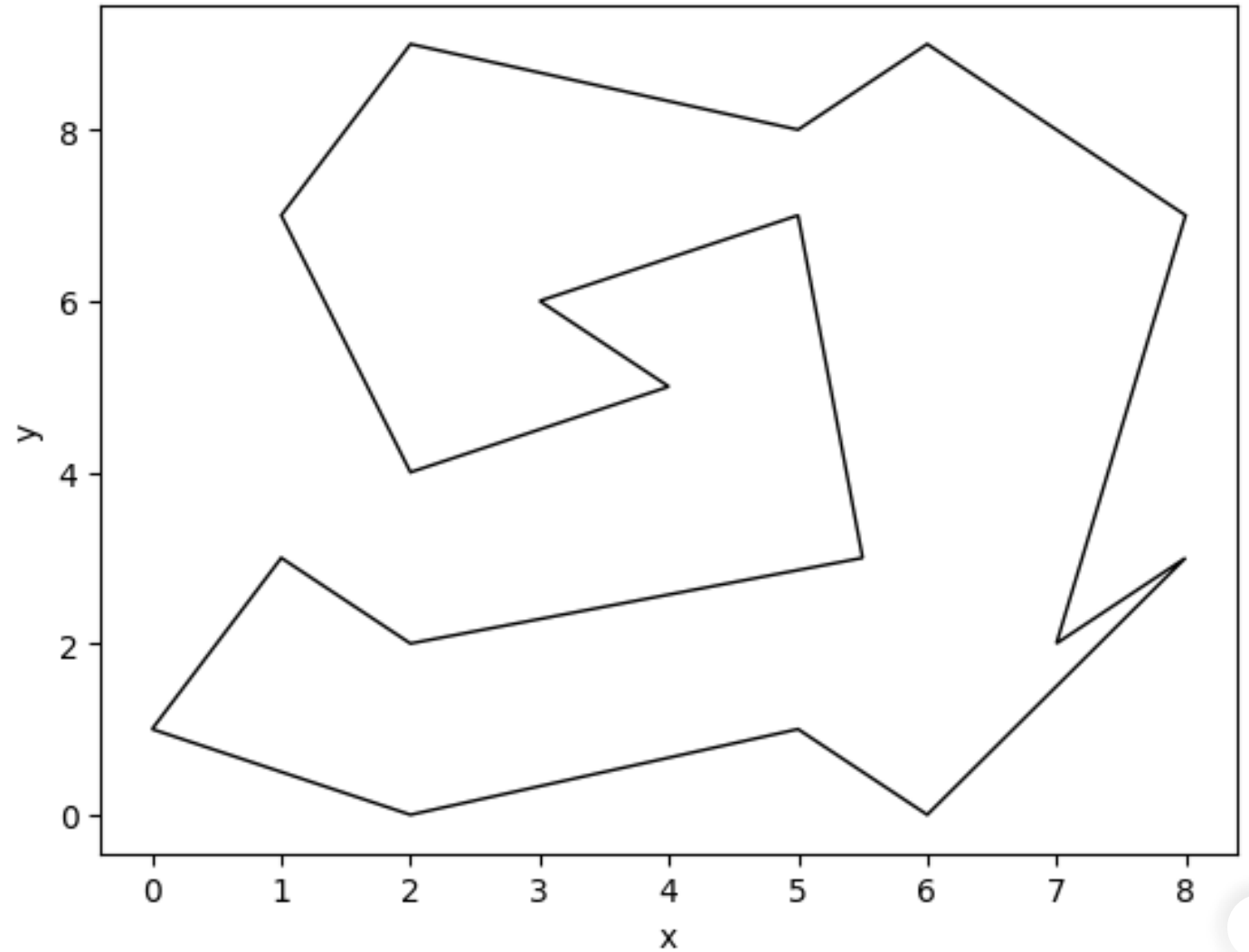
Metoda I

Triangulacja z wykorzystaniem wielokątów monotonicznych

Celem algorytmu jest znalezienie triangulacji dowolnego wielokąta prostego. Aby to osiągnąć w tym sposobie musimy przeprowadzić kilka kroków:

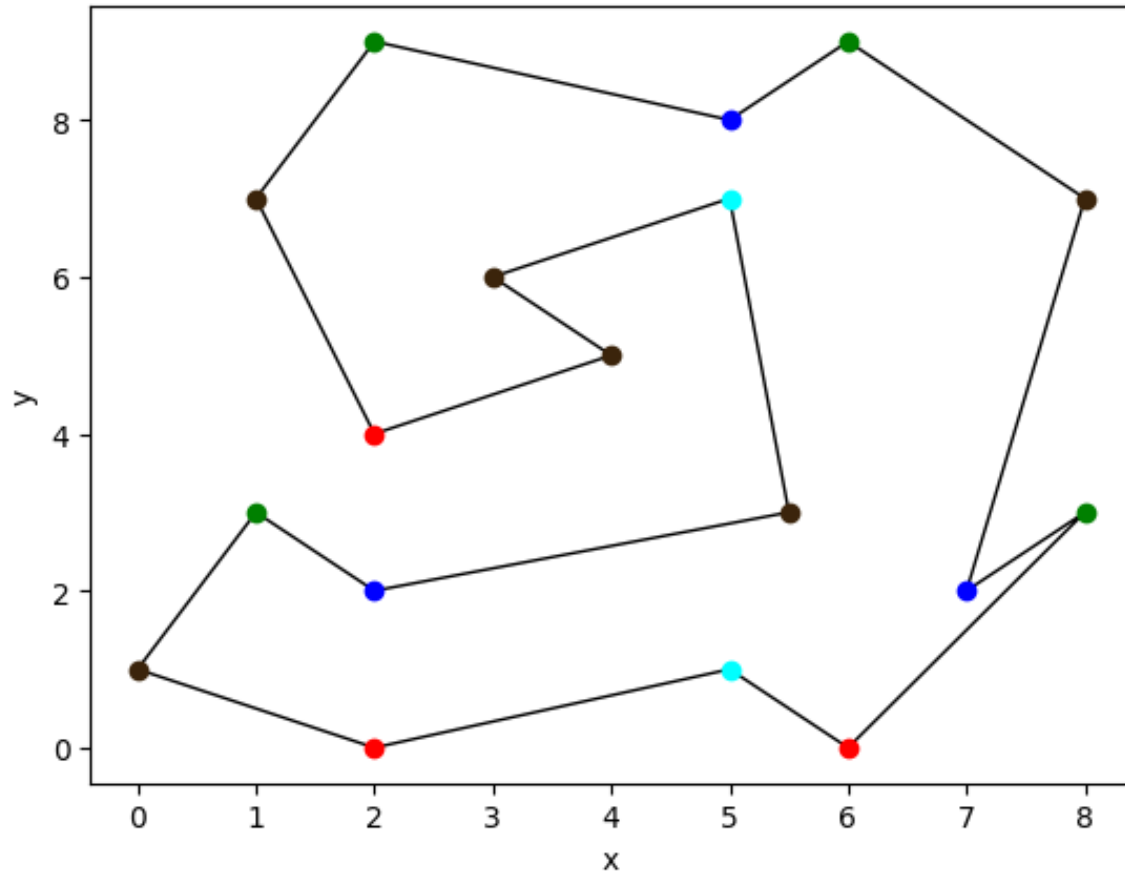
- + 1. Klasyfikacja wierzchołków danego wielokąta.
- + 2. Podział wielokąta na wielokąty monotoniczne.
- + 3. Triangulacja wielokątów monotonicznych.

Wielokąt na którym
przedstawimy
działanie



Klasyfikacja wierzchołków

- + Początkowy - kolor zielony
- + Końcowy - kolor czerwony
- + Łączący - kolor niebieski
- + Dzielący - kolor cyjan
- + Prawidłowy - kolor brązowy



Podział wielokąta na wielokąty monotoniczne

- + Q - struktura zdarzeń - kolejka priorytetowa - są to nasze punkty, gdzie największy priorytet ma górny lewy punkt
- + T - struktura stanu - dynamiczne drzewo wyszukiwań binarnych - są to krawędzie posortowane od lewej strony do prawej w miejscu przecięć z miotłą

Przebieg algorytmu

- + Kolejne punkty z kolejki priorytetowej Q będziemy przetwarzać w sposób zależny od typu danego punktu.
- + Głównym celem jest wyeliminowanie wierzchołków łączących oraz dzielących. Wielokąt y -monotoniczny to takich, który właśnie nie posiada żadnych wierzchołków tego typu.

Wierzchołek dzielący

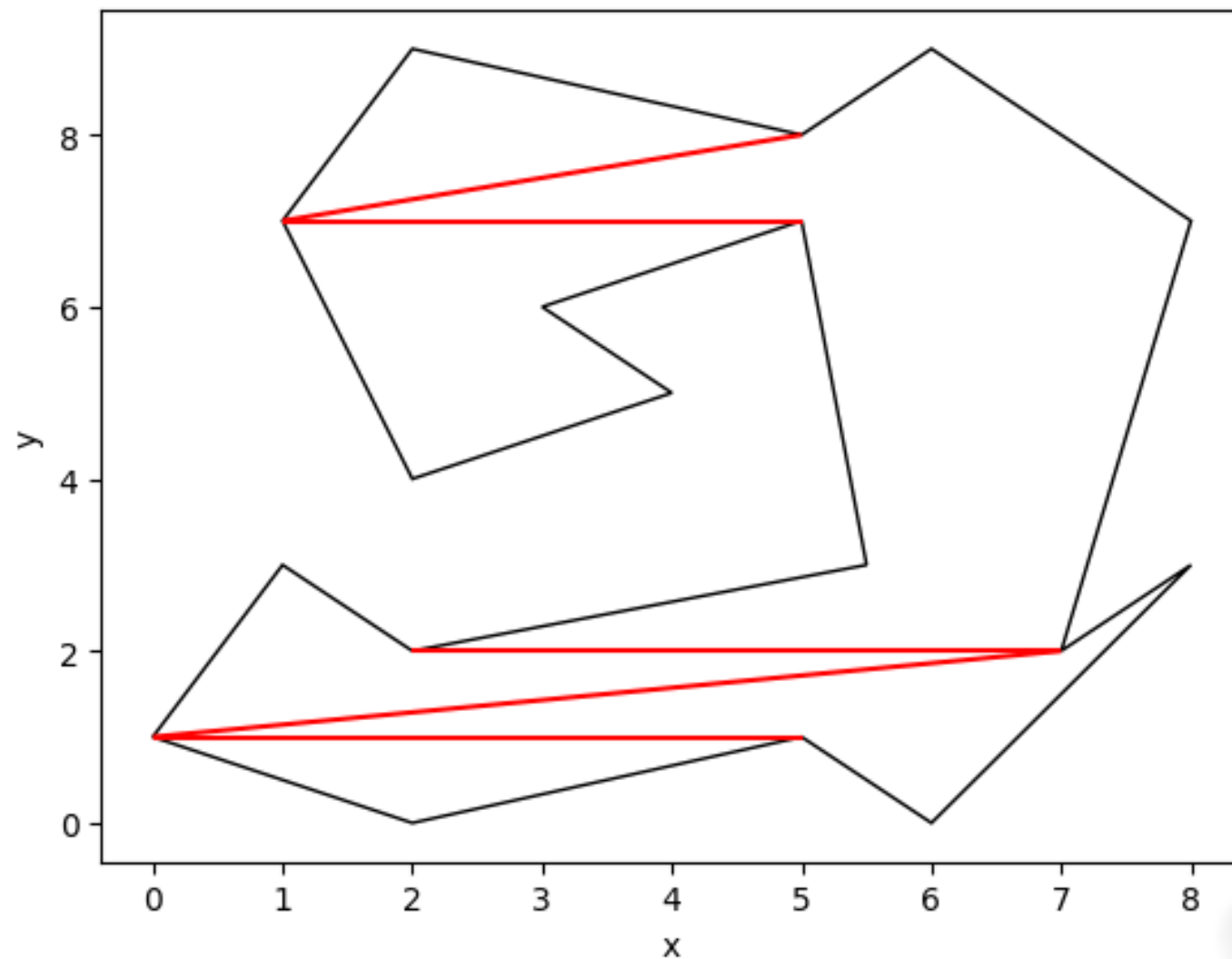
Będziemy starali się znaleźć wierzchołki znajdujące się w pobliżu rozpatrywanego aktualnie wierzchołka v_i . Aby szybko znajdować taki wierzchołek dla v_i dzielącego zauważmy, że będzie on najniższym wierzchołkiem leżącym pomiędzy prostą e_l będącą bezpośrednio na lewo od v_i oraz e_p będącą bezpośrednio po prawej stronie. Wierzchołek ten nazwiemy pomocnikiem(v_i), zauważmy, że może on być górnym końcem samej e_l .

Wierzchołek łączący

Pozostają jeszcze wierzchołki v_i łączące. Potrzebować one będą wierzchołków znajdujących się pod miotłą (jeszcze nie przetworzonych), ale będzie to dokładnie odwrócona sytuacja dla wierzchołka dzielącego. Za pomocnika(v_i), będziemy wybierać wierzchołek leżący pomiędzy e_l a e_p , znajdujący się najbliżej.

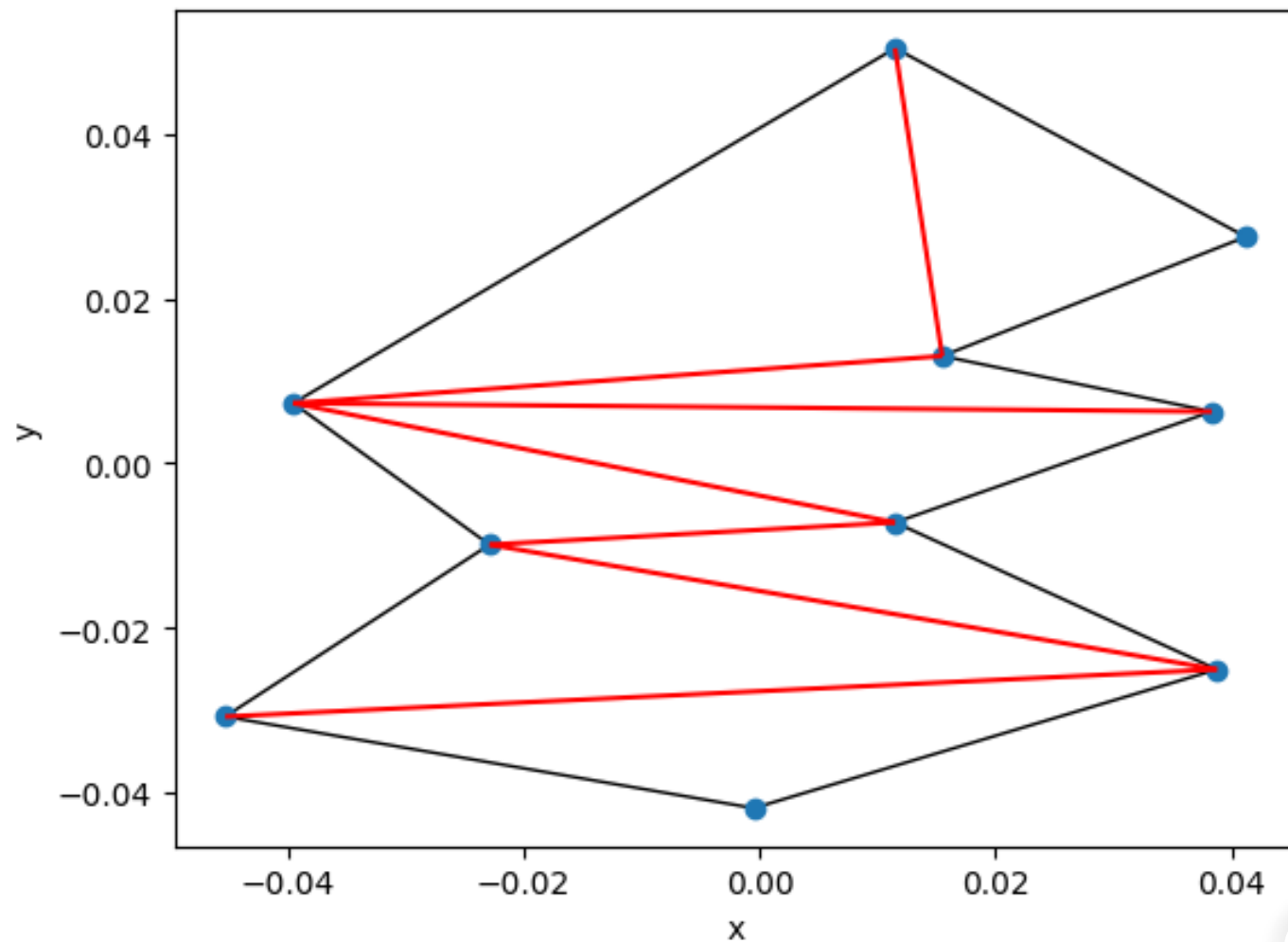


Rezultat podziału





Triangulacja wielokątów monotonicznych



Konwersja punktów i przekątnych na poszczególne wielokąty y-monotoniczne

Sporządzimy listę punktów, jako strukturę gdzie każdy punkt posiada zbiór innych punktów z których możemy do niego dotrzeć na podstawie otrzymanych przekątnych.

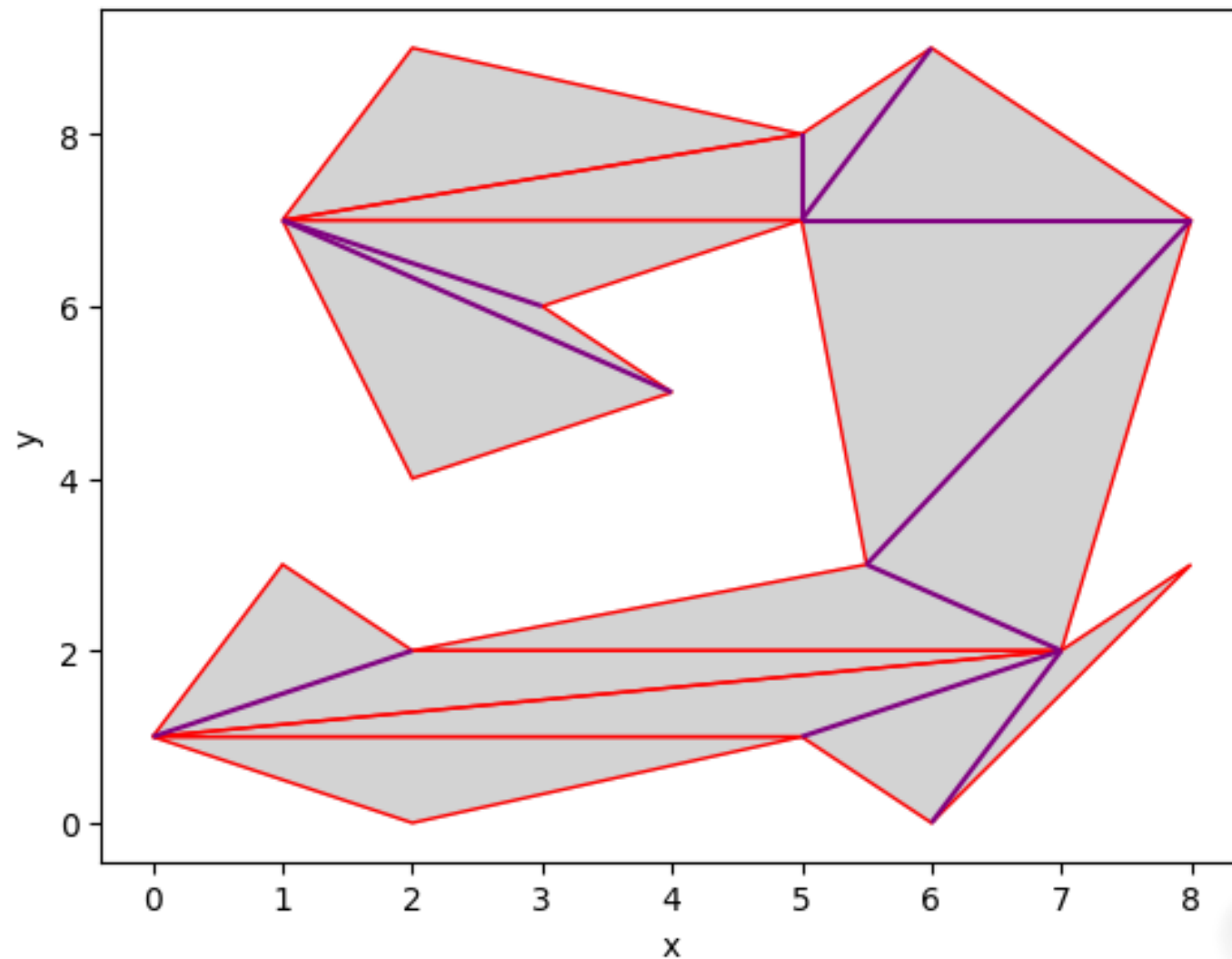
Posłużymy się następnie lekko zmodyfikowanym algorytmem przeszukiwania wzdłuż. Dla każdego punktu będziemy szukać ścieżek w których wracamy do punktu startu. Jedyne na co musimy uważać, to aby od razu nie wrócić się do tego samego punktu. Po znalezieniu takiej ścieżki usuwamy połączenia ze struktury i próbujemy dla kolejnego punktu.

Triangulacja kolejnych wielokątów y-monotonicznych

- + Określamy lewy i prawy łańcuch wielokąta względem kierunku monotoniczności
- + Porządkujemy wierzchołki wzdłuż kierunku monotoniczności
- + Wkładamy dwa pierwsze wierzchołki na stos.
- + Jeśli kolejny wierzchołek należy do innego łańcucha niż wierzchołek stanowiący szczyt stosu, to możemy go połączyć ze wszystkimi wierzchołkami na stosie. Na stosie zostają dwa wierzchołki, które były „zamiatane” ostatnie.
 - Jeśli trójkąt należy do wielokąta, to dodajemy przekątną i usuwamy odpowiedni wierzchołek ze szczytu stosu
 - w przeciwnym przypadku umieszczamy badane wierzchołki na stosie.



Rezultat triangulacji



Kompletny algorytm

- + Dane wejściowe
 - Wielokąt prosty zadany w postaci punktów w kierunku przeciwnym do wskazówek zegara.
- + Rezultat
 - Triangulacja wielokąta prostego w postaci przekątnych, które należy dołożyć.
- + Złożoność czasowa kompletnego algorytmu dla n wierzchołków: $O(n \cdot \log(n))$
- + Złożoność pamięciowa kompletnego algorytmu dla n wierzchołków: $O(n)$

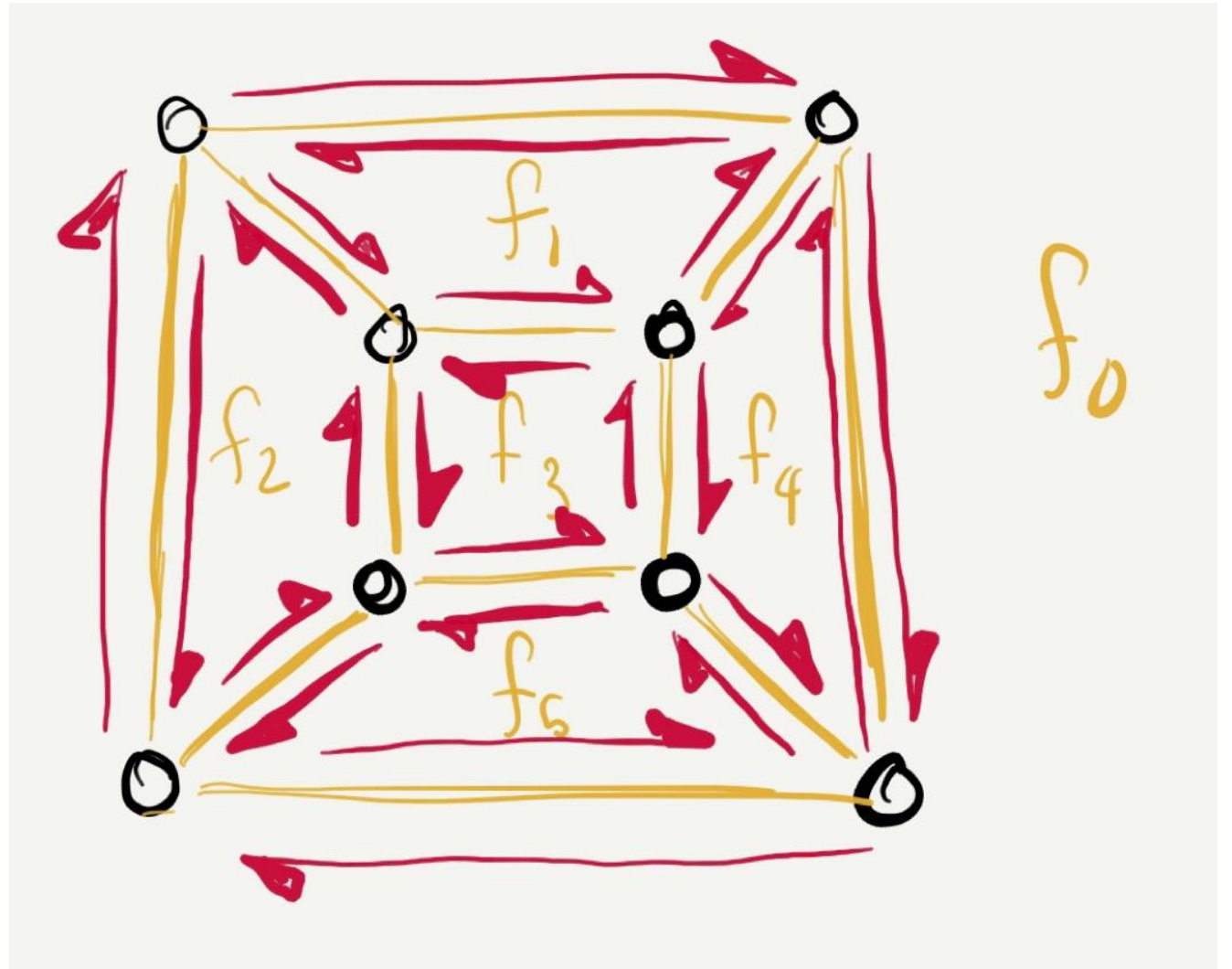
Metoda II Triangulacja Delaunay'a

Triangulacja Delaunay'a

- + Triangulacja Delaunay'a jest to triangulacja zbioru punktów na płaszczyźnie 2D taka, że żaden punkt nie leży wewnątrz okręgu opisanego na trójkącie należącym do triangulacji. Triangulacja Delaunay'a jest używana m.in. do modelowania terenu, automatycznego generowania meshu.
- + Dalej rozpatrywany algorytm to triangulacja Delaunay'a z ograniczeniami (z procedurami odzyskania krawędzi oraz usuwania trójkątów zewnętrznych). (triangulacja wieloboku)

Struktura danych do przechowywania siatki triangulacji

- + Do reprezentacji siatki użyta została struktura Half-Edge posiada on następujące atrybuty:
- + origin – punkt z którego półkrawędź wychodzi
- + next – następnik (też Half-Edge)
- + prev - poprzednik (też Half-Edge)
- + face – ściana, której jest częścią
- + twin – obiekt Half-Edge zwrócony przeciwnie na tej samej krawędzi





1. Wstępna
triangulacja



2.
Odzyskiwanie
krawędzi

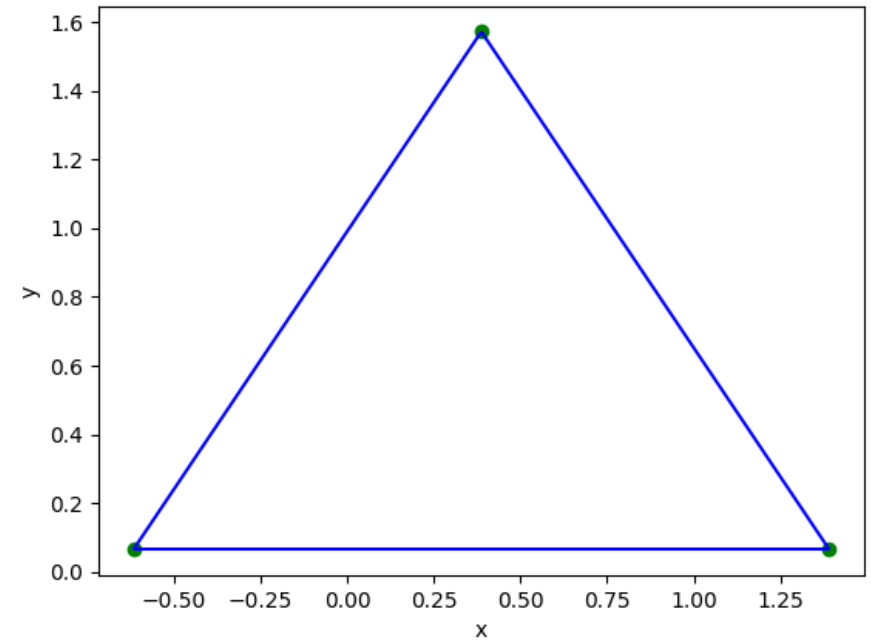


3. Usuwanie
trójkątów
zewnętrznych

Etapy algorytmu

1. Wstępna triangulacja

- + Mając triangulację n punktów dokładamy kolejny punkt, uwzględniając warunek okręgu (w okręgu opisanym na trójkącie nie może się zawierać żaden inny punkt) otrzymujemy triangulację dla $n+1$ punktów.
- + Jako początkowy stan triangulacji przyjmujemy trójkąt zawierający wszystkie punkty zbioru, zatem jest to triangulacja dla $n=3$ punktów.





Wstawienie punktu

1. Znalezienie trójkąta, w którym zawiera się punkt.

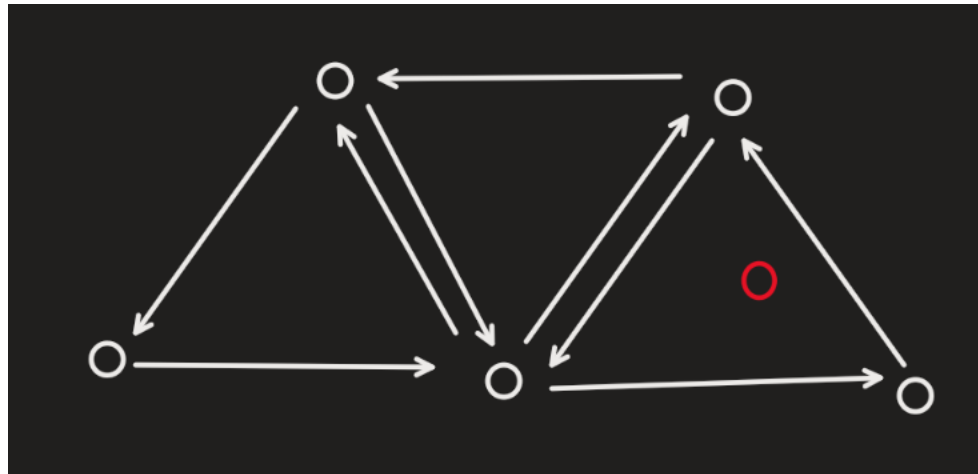
2. Znalezienie trójkątów, których okręgi opisane zawierają wstawiony punkt.

3. Usunięcie znalezionych trójkątów.

4. Utworzenie nowych trójkątów spełniających warunki triangulacji Delaunay'a.

Wyszukanie trójkąta, w którym zawiera się punkt

- + Wyszukiwanie odbywa się przy pomocy funkcji `orient()`, która przyjmuje półkrawędź oraz dany punkt. Półkrawędzie są skierowane otrzymujemy, zatem informacje czy punkt leży po prawej czy po lewej danej półkrawędzi. Każdy trójkąt reprezentowany jest przez trzy półkrawędzie skierowane przeciwnie do wskazówek zegara. Jeśli funkcja `orient` dla danego trójkąta zwróci dodatnią wartość (punkt leży po lewej) dla wszystkich półkrawędzi to oznacza iż rozpatrywany trójkąt jest szukany. Gdy dla którejkolwiek z półkrawędzi punkt leży po prawej porzucamy aktualnie rozpatrywany trójkąt i przechodzimy do sąsiada sąsiadującego rozpatrywaną krawędzią.



Wyszukiwanie trójkątów, których otoczki zawierają dany punkt

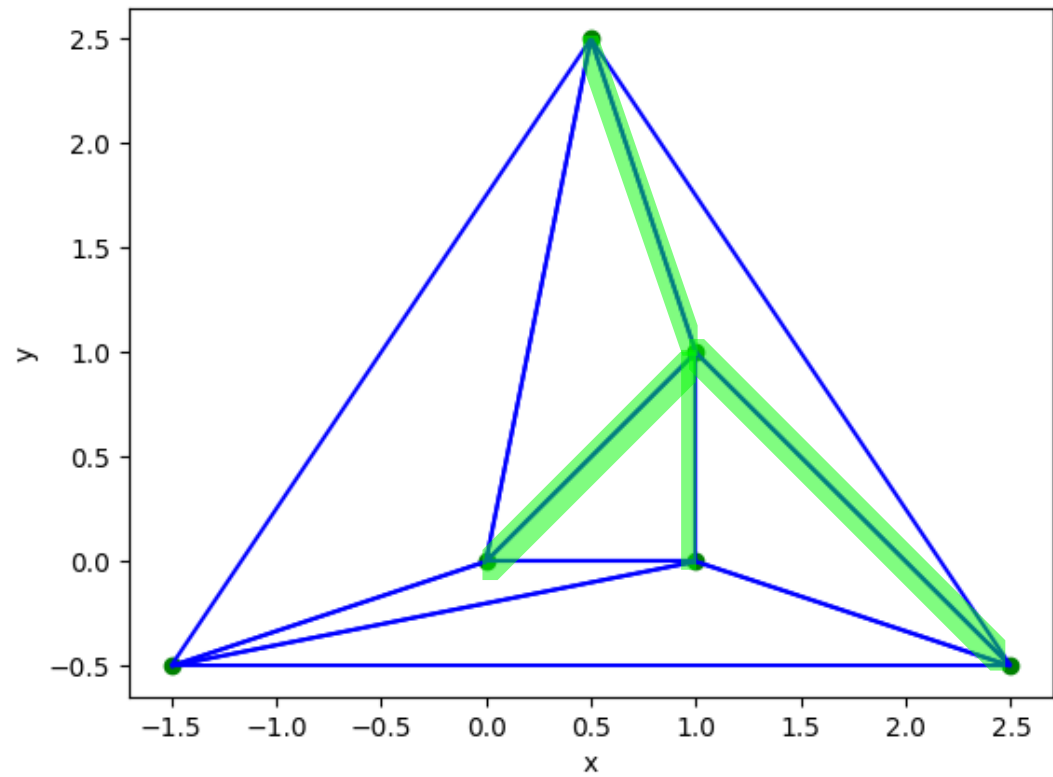
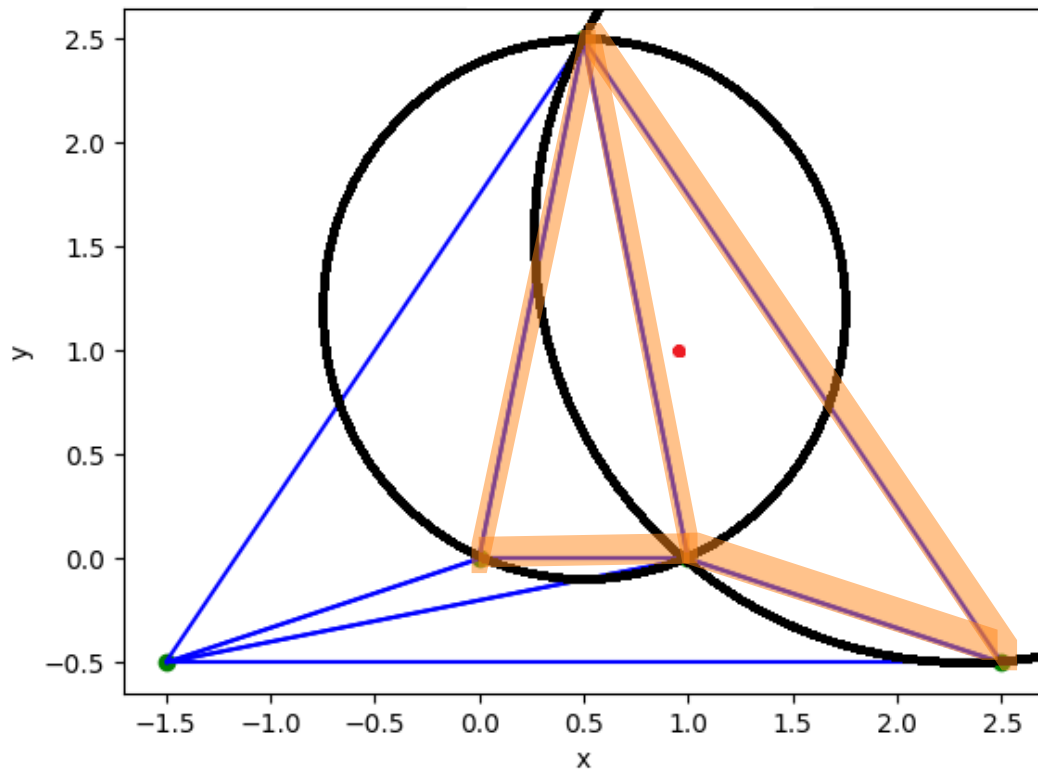
- + Po znalezieniu trójkąta zawierającego wstawiany punkt przy pomocy przeszukiwanie w szerz (BFS) szukamy wszystkich trójkątów, które w swej otoczce zawierają wstawiany punkt.

- + Realizacja BFS:

Jeśli dany trójkąt w swej otoczce zawiera dany punkt, do kolejki dodawane są wszystkie trójkąty będące sąsiadami rozpatrywanego, algorytm wykonuje się dopóki wszystkie trójkąty zawierające w swym okręgu opisanym wstawiany punkt nie zostaną przetworzone.

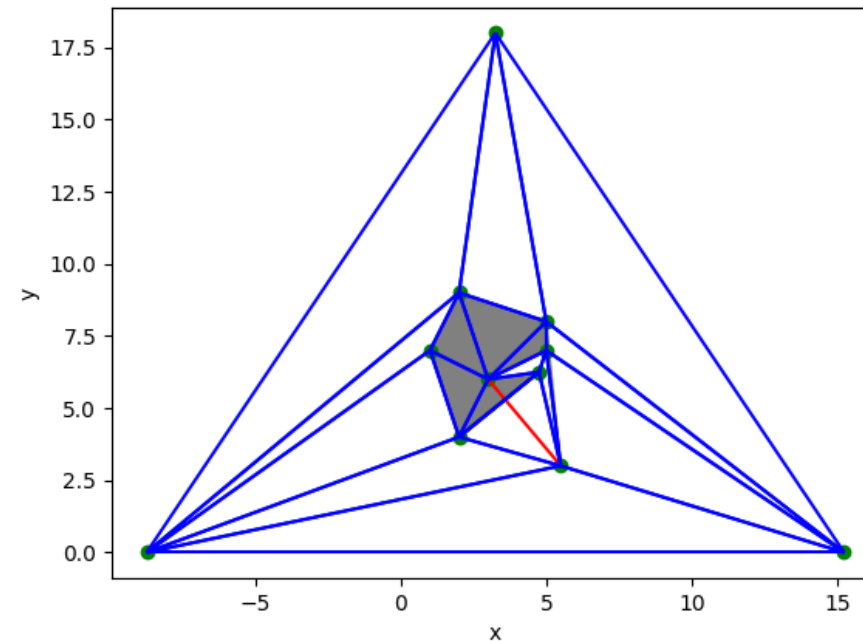
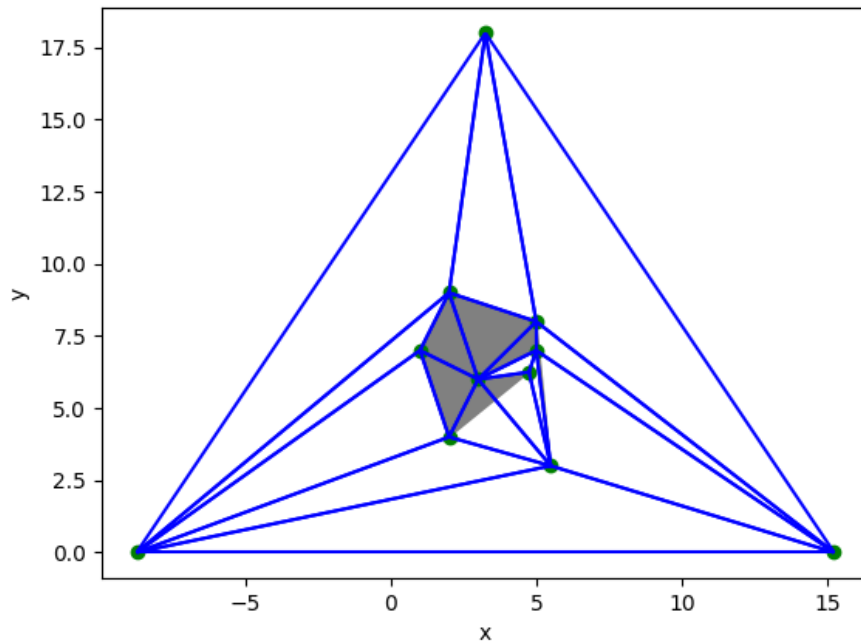
Graficzna reprezentacja dodawania punktu

Na pierwszym rysunku widać okręgi opisane na dwóch trójkątach, w których zawiera się wstawiany punkt. Następnie zostały one usunięte i zastąpione trójkątami utworzonymi poprzez połączenie dodawanego punktu z punktami będącymi wierzchołkami wielokąta utworzonego przez usunięte trójkąty.



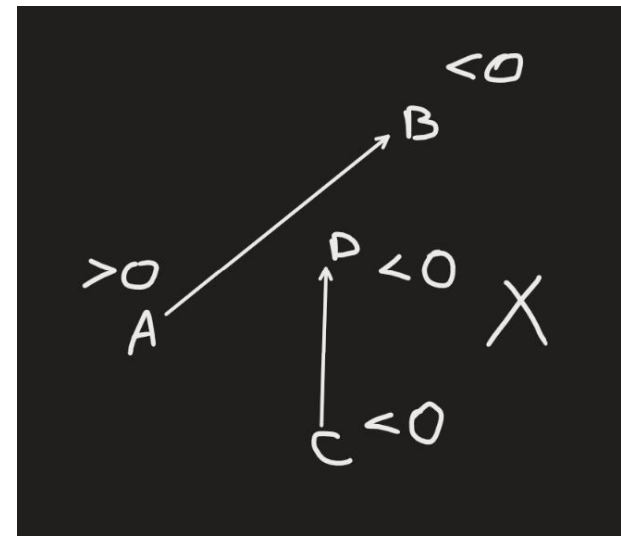
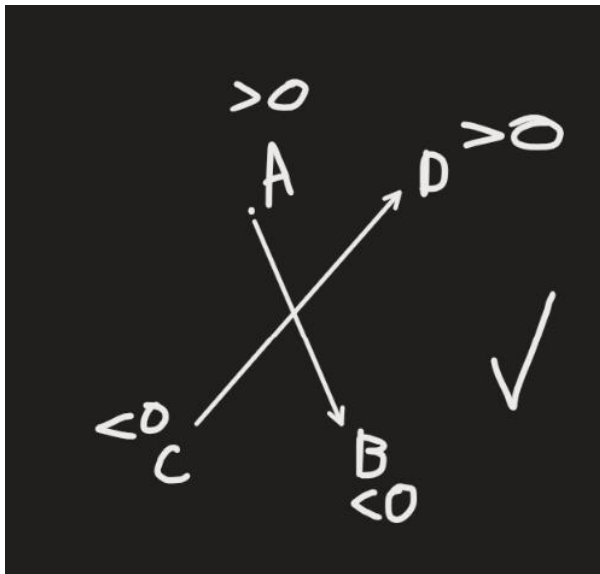
2. Odzyskiwanie krawędzi

- Po pierwszym etapie otrzymujemy triangulację Delaunay'a, jednak nie uwzględnia ona boków wielokąta, którego triangulację chcieliśmy otrzymać. Zatem następnym etapem będzie modyfikacja triangulacji, aby była ona triangulacją wielokąta, a nie tylko triangulacją zbioru punktów.



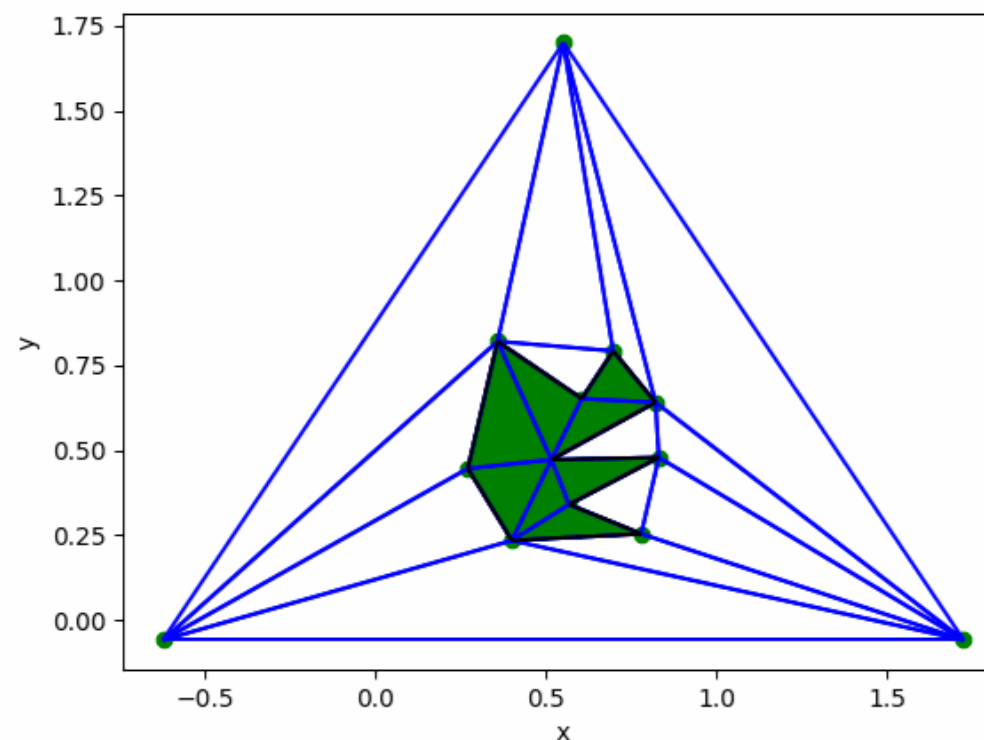
Wykrywanie przecięć

- + Aby uzyskać triangulację wielokąta, iterujemy po każdej jego krawędzi i sprawdzamy, czy jest ona zawarta w powstałej triangulacji, jeśli nie, wyszukujemy krawędzie triangulacji, które ją przecinają i zastępujemy je.
- + Samo wykrywanie przecięć realizowane jest również za pomocą funkcji `orient()` zwracającej liczbę dodatnią, jeśli punkt znajduje się po lewej stronie półkrawędzi, a ujemną jeśli po prawej. Zatem warunkiem koniecznym, aby dwie krawędzie się przecinały jest ujemny iloczyn wyników wywołań `orient` dla danej półkrawędzi, mianowicie dla punkt C i D względem półkrawędzi AB oraz A i B względem CD.

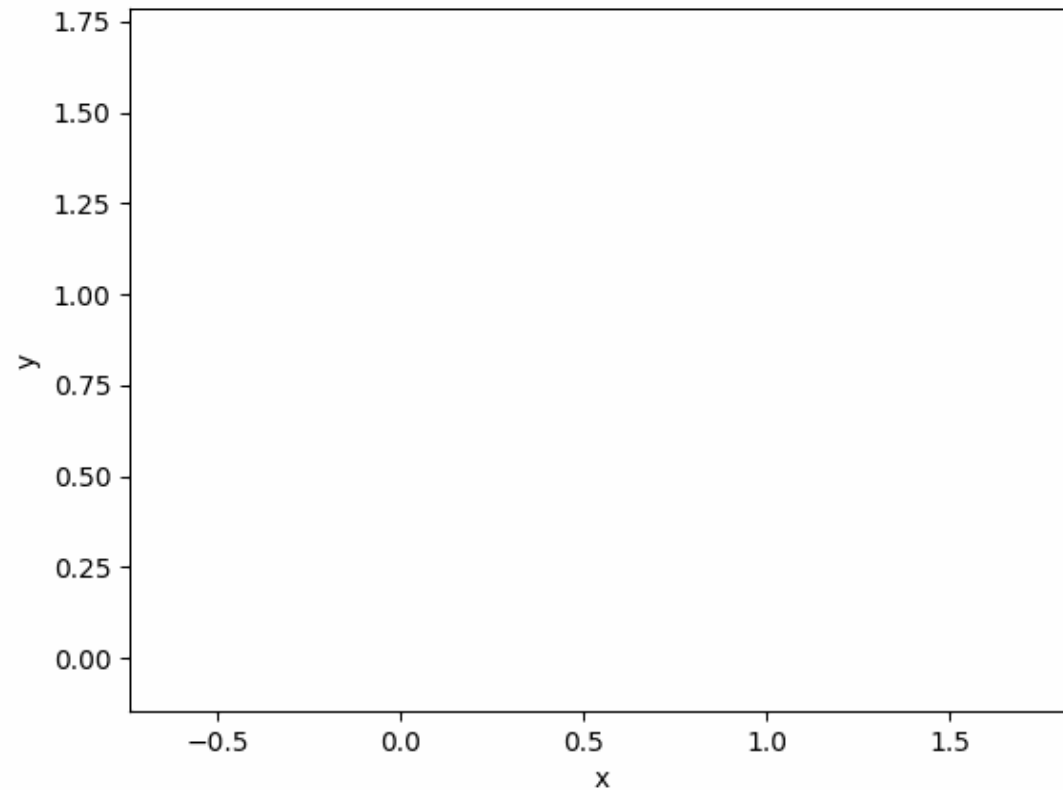


3. Usuwanie trójkątów zewnętrznych

- + Wielokąt zadany jest przeciwnie do wskazówek zegara zatem biorąc półkrawędź z pierwszego do drugiego wierzchołka wiemy, że trójkąt, do której ona (ta półkrawędź) należy znajduje się wewnątrz wielokąta. Potem poruszając się po kolejnych sąsiadach rozpatrywanego trójkąta (możemy wejść do sąsiada pod warunkiem, że krawędź ich łącząca nie należy do wieloboku, bo wtedy wyszlibyśmy poza wielobok) oznaczamy trójkąty jako „odwiedzone”. Pozostałe trójkąty usuwamy, w ten sposób otrzymujemy ostateczną triangulację Delaunay’a. Na rysunku wszystkie trójkąty poza oznaczonymi na zielono należącymi do wieloboku są do usunięcia

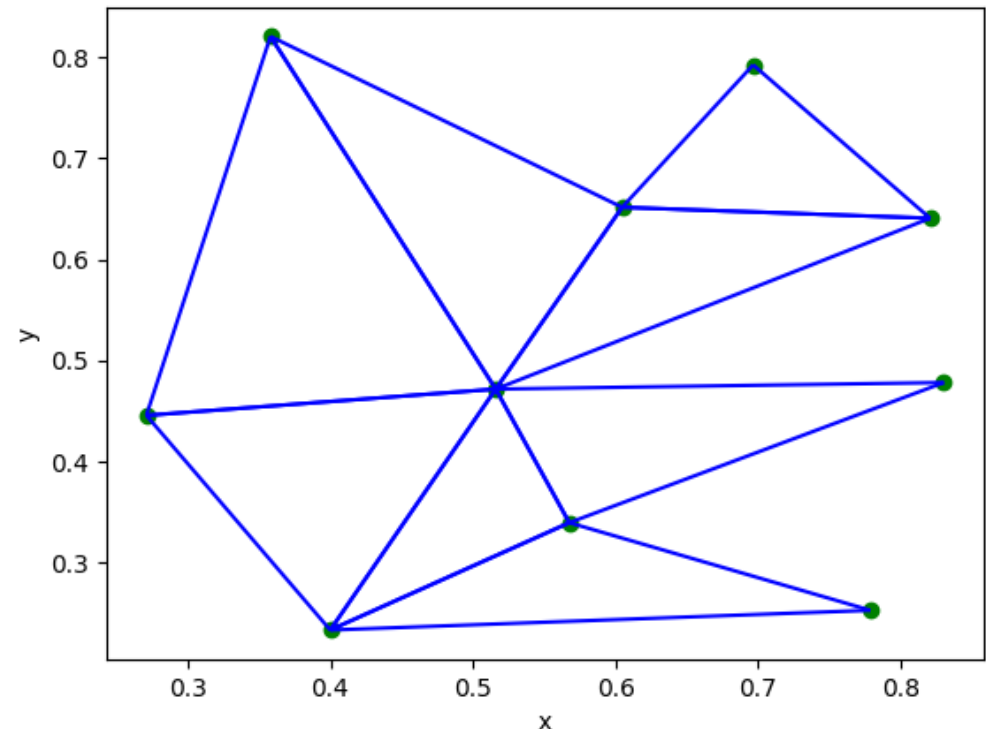


Wizualizacja poszukiwania trójkątów wewnątrz przykładowego wieloboku



Usuwanie nieodwiedzonych trójkątów

- + Ostatnim etapem triangulacji Delaunay'a wieloboku jest usunięcie trójkątów triangulacji, nie zawierających się w wieloboku. Odbywa się to iterując po trójkątach triangulacji, jeśli dany nie został odwiedzony to usuwany jest z ostatecznej triangulacji.
- + Ostateczna postać triangulacji dla przykładu z poprzedniego slajdu znajduje się obok.



Testy



Dziękujemy za uwagę!

Źródła:

- Wykłady z Algorytmów Geometrycznych dr inż. Barbary Głut
- Wikipedia
- [Half-Edge Data Structure](#)
- [Half_Edges_2](#)

Autorzy:
Robert Mesek
Wojciech Kaźmierczak